ENCE360 Assignment 2026

Due at 18:00 Fri 09/10

Introduction

This assignment comes with a single program, `serial.c`, that completes a menial task - numerical integration. As is, the program prompts the user for a number range, a number of slices, and the index of the function it should integrate over this range.

We will modify this file in two main ways:

- We will multithread the integration. This will simply make it run faster.
- After reading the user's prompts, the calculation will be handled by a child process. The parent process will take user requests, and spawn a child process to handle the task. This is similar to how an operating system's terminal works.

We will then combine these two modifications together, and run some performance experiments on the different program versions.

All your work for this project must be done in a repository on eng-git. You will be assigned a repository of the form `ence360-assignment-2026/abc123`, and must use it for your writing/experiments. Commit regularly! Your commit history is proof of your development and experimentation, and will be marked.

Notes:

- Compile all programs with the `-lm` and `-lpthread` flags on the end.
- All programs should compile with `-Wall -Werror`, so fix those warnings!

1. Multithreaded Execution

Your first task is to make a faster version of the supplied program. Make a copy of the original `serial.c`, this time named `thread.c`.

Your program should contain a `#define` for the number of threads to split the work between. Split the work of the integration as equally as possible between a fixed number of threads by splitting up the range. For example, the prompt

```
gauss -1 1 16
```

run with 4 worker threads should split the 16 slices evenly, 4 slices per thread. The prompt

```
gauss -1 1 18
```

run with 4 worker threads should have two 4-slice threads and two 5-slice threads. You can ignore the case where there are fewer slices than threads.

You ***must*** declare a variable within `main()` that all threads write to before exiting, the access to which is controlled with a mutex. While this is not strictly necessary for a simple integration, it is for the general case where the threads take different times to execute, or the computation has more complex results.

I suggest you define a `struct` containing the arguments we want each thread to have access to, and make an array of these within your `main()` function. The `integrateTrap()` function will need to be modified to take a pointer to this `struct` as its argument, as a `void*`.

Be sure to check your implementation for correctness. You should notice a significant speedup roughly proportional to the number of threads used - e.g., using 4 threads, the program should run 4 times faster than `serial.c`.

2. Spawning Task Processes

Make another copy of the original `serial.c`, this time named `process.c`. This program will run the (serial) function `integrateTrap()` inside a child process. The parent process should then re-prompt for input. This means you'll be able to have several queries running at once. Your code should have a maximum number of worker processes - if the number of children is already at this maximum, it should not prompt for input until one of the children finishes. Your processes must all exit cleanly.

As a starting point, I suggest you `#define` a `MAX_CHILDREN` macro, and declare a `static` global semaphore `numFreeChildren`. The next step would be to register a signal handler that responds appropriately when a child completes - if you use `signal()` rather than `sigaction()`, be sure to do so safely. There are more steps beyond this, but hopefully it's enough to get you started. Your solution ***must*** use a semaphore and a signal handler.

3. Multithreaded Worker Processes

The final step is to combine these two modifications into a third new file, `processThread.c`. This should allow multiple jobs to be run at once, *and* be faster than `process.c`, especially when handling one task at a time.

This one should only require a few modifications given the code you've already written. Again, make sure you check for correctness by comparing it to the supplied code's answers.

4. Report

Once you've written these three programs, also write a short, 3-4 page report (excluding title/references) describing and explaining the performance differences you see between them. This will require running performance experiments. To do so, you'll want to measure how long the program takes to run without the

delay caused by a human typing in the tasks. You can do this by running the program with an input file in as follows:

```
time ./a.out < testCases.txt
```

You should test with more than just the sample `testCases.txt` provided. Your report should describe at least three well-structured experiments you conducted, exploring how program performance changes in different conditions. Each experiment should have at least one graph showing its results. The main purpose of this report is for you to show your understanding of the observed performance differences, especially quirks.

You also ***must*** include a few reputable references (in IEEE format) backing up your explanations of the observed behaviour. These references should be correct, relevant, and authoritative. References must be specific enough to actually find what you're citing. For example, citing a textbook usually requires referencing the section / page number.

Depending on the computer you're running your tests on, varying both the number of threads and the maximum number of child processes may produce very different results, which could be good to talk about.

Avoid testing your code on a virtual machine (for example, though WSL on Windows) - running the program natively (ideally on the Linux lab machines) will give you more concrete information to discuss in your report.

Your experimental work and report writing must also be done with regular commits on git - think of it as a lab book. This lets me ensure you've gone about the experimentation rigorously, makes sure you won't lose your results, and lets me give you partial marks when appropriate.

5. Submission

You will submit your report (source documents as well as a PDF version), your experimental data, and your source files for this assignment. All of these should have been developed inside your assigned repository, so that I can see your work history. Make sure your code is laid out sensibly and is easy to follow. Any other working files produced during the assignment can and should be in the repository, just make sure it's easy for me to find the report, source, and experiment data.

Late submissions will attract a 10% penalty per day overdue.

6. Generative AI

Generative AI (ChatGPT, Claude, Gemini, Deepseek, Llama, etc.) *may* be used, but nothing they generate should be directly submitted. Instead, use them (cautiously) to improve your understanding, and to help you debug. You should have manually written your submitted code and report yourself. There will be a question on the assignment in the exam, so don't be over-reliant.